Clase 6 · Mapa de memoria, Pico ID y aritmética en ensamblador Thumb

Organización y Arquitectura de Computadoras · M.I. José Antonio de Jesús Arredondo Garza · lunes 21 sep 2026 · Tema 3 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Mapa de memoria de datos y dirección efectiva

Hay que distinguir memoria de programa y memoria de datos. En la Pico cada periférico interno tiene una dirección fija en el espacio de datos.

Región	Dirección
RAM interna	desde ≈ 0x20000000
Periféricos internos	desde 0x40000000
Chip ID (Pico ID)	0x40000004 (base + 4), igual en Pico 1 y Pico 2

Flash: Pico 1 = 2 MB; Pico 2 = 4 MB.

> **Fórmula (examen)**
>
> **Dirección efectiva = Dirección base del periférico + Offset específico.** Leer 0x40000004 permite que el programa identifique el modelo y ramifique (if / branch) para no ejecutar instrucciones incompatibles.
>
> Error común: confundir memoria de programa con memoria de datos, o usar direcciones absolutas sin sumar el offset.

2. Identificar la placa con machine.mem32

`mem32` lee y escribe palabras de 32 bits directo en el mapa de memoria, sin librerías.

```
from machine import mem32
revision_addr = 0x40000000 + 4
revision = machine.mem32[revision_addr]
```

Modelo	Código
Pico 1	0x10002927
Pico W	0x20002927
Pico 2	0x10004927

En clase se conectó una Pico 1 y luego una Pico 2; el programa imprimía el modelo correcto.

3. La misma lectura en ensamblador inline (@micropython.asm_thumb)

Registros generales R0–R15. No se puede cargar un inmediato de 32 bits (0x40000000) en una instrucción: el compilador marca error. Truco con desplazamiento lógico:

1. `mov(R1, 0x40)` — carga el byte.
2. `lsl(R1, R1, 24)` — lo corre 24 bits: R1 = 0x40000000.
3. `ldr(R0, [R1, 0])` — lectura indirecta de la dirección apuntada por R1 (con offset si hace falta).

> **Regla de retorno**
>
> Toda subrutina en ensamblador inline **devuelve a Python el contenido de R0**. `val = id_func()` y luego `print(hex(val))`.
>
> Error común: intentar `mov` con una constante de 32 bits (o de 16, como 0x4000).

4. Aritmética: multiplicación y suma

- Parámetros: deben ser **registros consecutivos empezando en R0** (`def f(r0, r1, r2)`); empezar en R3 o saltarse registros da error.
- `mul` en Thumb usa **dos operandos**: `mul(R1, R2)` → R1 = R1 × R2.
- Inmediatos limitados a **8 bits (0–255)**; valores mayores se pasan como variables de Python por los argumentos.

Ejercicio resuelto: multiplica(10, 50) = 500

```
@micropython.asm_thumb
def multiplica(r0, r1):
    mov(R2, R1)
    mov(R1, R0)
    mul(R1, R2)   # 10 * 50 = 500
    mov(R0, R1)   # resultado a R0
```

Suma de cuatro operandos (`def suma(r0, r1, r2, r3)`): `add(R2, R2, R3)`, `add(R1, R1, R2)`, `add(R0, R0, R1)` → resultado en R0.

Error común: `mul(R0, R1, R2)` con tres operandos.

5. Tarea: raíz cuadrada entera por restas de impares

Restar 1, 3, 5, 7, 9… mientras el resultado no sea negativo; el número de restas válidas es la raíz.

Caso	Restas	Resultado
√25	25−1=24, −3=21, −5=16, −7=9, −9=0	raíz 5, exacta
√27	27−1=26, −3=23, −5=18, −7=11, −9=2; 2−11 = −9 ✗	raíz 5, residuo 2

Extensión opcional: decimales multiplicando el operando por potencias pares de 10 (100, 10000…).
Próxima clase: GPIO y temporizadores con mem32 y Thumb.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
Mapa de datos	RAM 0x20000000; periféricos 0x40000000
Dirección efectiva	base + offset
Chip ID	0x40000004 → Pico 1 0x10002927, Pico W 0x20002927, Pico 2 0x10004927
Cargar 0x40000000	`mov(R1, 0x40)` + `lsl(R1, R1, 24)`; leer con `ldr(R0, [R1, 0])`
Parámetros	registros consecutivos desde R0
Retorno	siempre R0
mul	dos operandos: R1 = R1 × R2
Inmediatos	máximo 8 bits (0–255)

Ejercicio tipo examen

√27 por impares: 1, 3, 5, 7, 9 caben (queda 2); el 11 ya no → raíz 5, residuo 2.